

ROSA Technical Solutions

Here is the scenario-by-scenario architectural breakdown and technical solution for running both Kubernetes-native and Non-Kubernetes (VM) workloads on Red Hat OpenShift Service on AWS (ROSA).

Part 1: Kubernetes-Only Workloads

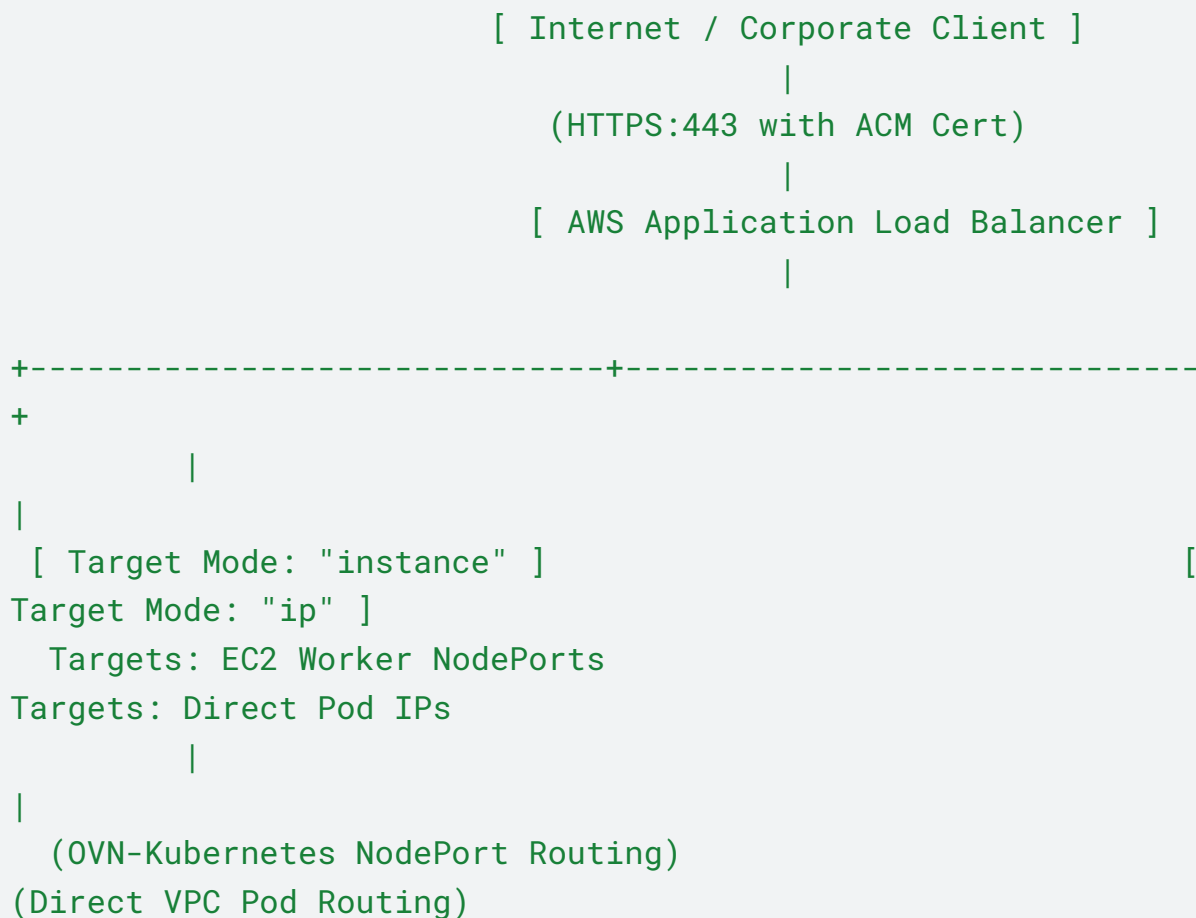
Scenario 1: ALB Ingress via AWS Load Balancer Controller with ACM Cert (Instance vs. IP Mode)

1. Summary of Ask

Expose an HTTP application on ROSA using an AWS Application Load Balancer (ALB) via the AWS Load Balancer Controller, securing it with an AWS Certificate Manager (ACM) public TLS certificate. Compare Instance mode (routing via EC2 NodePorts) versus IP mode (routing directly to Pod IPs with Pod Readiness Gates).

2. Architecture Diagram

None



```

|
|
| [ Application Pod ]
[ Application Pod ]
+ Pod
Readiness Gate Check

```

3. Technical Solution & Mechanics

- Instance Mode (`target-type: instance`): Traffic flows from the ALB to an AWS EC2 NodePort on worker nodes, after which OVN-Kubernetes routes traffic internally to the target Pod.
- IP Mode (`target-type: ip`): Traffic routes directly from the ALB to the Pod IP address across the VPC.
- Pod Readiness Gates: When using `ip` mode, the AWS Load Balancer Controller automatically injects a Pod Readiness Gate condition (`target-health.elbv2.k8s.aws/k8s-app`) into the Pod specification. During rolling deployments, Kubernetes will not mark new Pods as `Ready` until the ALB target group health check passes, eliminating dropped connections during updates.

None

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webserver-alb-ingress
  namespace: my-app
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    # ACM Public Certificate ARN
    alb.ingress.kubernetes.io/certificate-arn:
arn:aws:acm:us-east-1:123456789012:certificate/abc12345-1234-1
234-1234-abcdef123456
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'
    # Target Mode selection ('instance' or 'ip')

```

```

      alb.ingress.kubernetes.io/target-type: ip
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: webserver-svc
            port:
              number: 80

```

- Official Documentation & Guides:
 - [Adding a Private Ingress Controller and a Public ALB to a ROSA Cluster](#)
 - [ROSA Best Practices: Private clusters and landing-zone ingress](#)

Scenario 2: Pod Authentication using IRSA (STS AssumeRoleWithWebIdentity)

1. Summary of Ask

Configure a Kubernetes workload to interact with AWS services (e.g., read/write to an Amazon S3 bucket) using IAM Roles for Service Accounts (IRSA) and AWS Security Token Service (STS) temporary web-identity credentials, avoiding long-lived static AWS access keys.

2. Architecture Diagram

None





3. Technical Solution

1. AWS IAM Role Trust Policy: Restricts role assumption to the cluster's OIDC issuer and the specific `ServiceAccount` namespace and name.
2. Kubernetes `ServiceAccount`: Annotated with `eks.amazonaws.com/role-arn`.
3. OpenShift Pod Injection: OpenShift automatically mounts a projected OIDC token to `/var/run/secrets/openshift/serviceaccount/token` and sets `$AWS_ROLE_ARN` and `$AWS_WEB_IDENTITY_TOKEN_FILE` inside the container. AWS SDKs automatically detect these environment variables to execute `sts:AssumeRoleWithWebIdentity` to retrieve temporary credentials.

None

```
// AWS IAM Role Trust Policy (app-s3-role)
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated":
"arn:aws:iam::123456789012:oidc-provider/rh-oidc.s3.us-east-1.
amazonaws.com/cluster-id"
      },

```

```

    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "rh-oidc.s3.us-east-1.amazonaws.com/cluster-id:sub":
"system:serviceaccount:my-app:s3-writer-sa"
      }
    }
  ]
}

```

None

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: s3-writer-sa
  namespace: my-app
  annotations:
    eks.amazonaws.com/role-arn:
arn:aws:iam::123456789012:role/app-s3-role
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: s3-uploader
  namespace: my-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: s3-uploader
  template:
    metadata:

```

```

labels:
  app: s3-uploader
spec:
  serviceAccountName: s3-writer-sa
  containers:
  - name: uploader
    image: amazon/aws-cli:latest
    command: ["sh", "-c", "aws s3 cp /etc/hostname
s3://my-app-data-bucket/test.txt && sleep 3600"]

```

- Official Documentation & Guides:
 - [ROSA Best Practices: Application workloads, IRSA, and STS](#)
 - [AWS STS and ROSA Workflow Explained](#)

Scenario 3: Persistent Storage using Amazon EBS gp3 PVCs

1. Summary of Ask

Provision persistent volume storage backed by Amazon EBS gp3 (which allows tuning IOPS and throughput independently of capacity) and mount it onto a Pod.

2. Architecture Diagram

None

```

[ Pod Spec ] ----> [ Volume Mount (/data) ]
                    |
                    [ PersistentVolumeClaim (gp3) ]
                    |
                    [ AWS EBS CSI Driver Operator ]
                    |
                    [ AWS EBS gp3 Volume ]

```

3. Technical Solution

ROSA comes pre-configured with the Amazon EBS CSI Driver Operator (openshift-cluster-csi-drivers-ebs). Persistent volumes created via EBS are Read-Write-Once (RWO) and encrypted by default.

None

```

apiVersion: storage.k8s.io/v1

```

```
kind: StorageClass
metadata:
  name: gp3-custom
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data-pvc
  namespace: my-app
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: gp3-custom
  resources:
    requests:
      storage: 20Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod
  namespace: my-app
spec:
  containers:
    - name: app
```

```
image: ubi9/ubi
command: ["sh", "-c", "echo 'data' > /data/test.log &&
sleep 3600"]
volumeMounts:
- mountPath: "/data"
  name: data-volume
volumes:
- name: data-volume
  persistentVolumeClaim:
    claimName: app-data-pvc
```

- Official Documentation & Guides:
 - [Red Hat OpenShift Service on AWS Storage Overview](#)

Scenario 4: User Workload Monitoring (PodMonitor & PrometheusRule)

1. Summary of Ask

Demonstrate how application metrics are scraped and alerting/recording rules are configured in OpenShift. Explain the OpenShift-native alternative to `kube-prometheus-stack`.

2. OpenShift Alternative & Mechanics

In OpenShift and ROSA, Red Hat SREs manage platform monitoring. To monitor developer application workloads, OpenShift provides User Workload Monitoring (UWM).

- Platform Setup: User Workload Monitoring is enabled by adding `enableUserWorkload: true` in the `cluster-monitoring-config` ConfigMap under the `openshift-monitoring` namespace.
- Developer Workloads: Once enabled, application teams deploy standard Prometheus Operator Custom Resources—such as `PodMonitor`, `ServiceMonitor`, and `PrometheusRule`—directly into their application namespaces without installing `kube-prometheus-stack`.

3. Technical Solution

None

```
# 1. Scrape definition via PodMonitor
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
```

```
metadata:
  name: webserver-pod-monitor
  namespace: my-app
  labels:
    app: webserver
spec:
  selector:
    matchLabels:
      app: webserver
  podMetricsEndpoints:
    - port: http-metrics
      path: /metrics
      interval: 15s
```

2. Custom Recording & Alerting Rules via PrometheusRule

```
apiVersion: monitoring.coreos.com/v1
```

```
kind: PrometheusRule
```

```
metadata:
  name: webserver-alert-rules
  namespace: my-app
```

```
spec:
  groups:
    - name: webserver.rules
      rules:
        # Custom Recording Rule
        - record: job:http_requests:rate5m
          expr: sum(rate(http_requests_total[5m])) by (job,
namespace)

        # Custom Alerting Rule
        - alert: HighErrorRate5XX
          expr: sum(rate(http_requests_total{status=~"5.."}[5m]))
/ sum(rate(http_requests_total[5m])) > 0.05
```

```

for: 2m
labels:
  severity: critical
annotations:
  summary: "High 5XX error rate detected on {{
$labels.namespace }}"
  description: "5XX HTTP requests exceed 5% over a
5-minute window."

```

- Official Documentation & Guides:
 - [Configuring User Workload Monitoring in OpenShift](#)

Scenario 5: One-Off GPU Worker Node on EC2 VM in a Bare Metal ROSA Cluster

1. Summary of Ask

Assuming the primary worker pool uses baremetal nodes (e.g., `m5zn.metal` for OpenShift Virtualization), demonstrate adding a singular GPU-enabled EC2 VM worker node (e.g., `g4dn.xlarge` or `g5.xlarge`) via a MachinePool, isolating it with taints, and scheduling a GPU workload onto it.

2. Architecture Diagram

None



Taint + NodeSelector)

(Tolerates

3. Technical Solution

Step 1: Create GPU MachinePool with ROSA CLI:

None

```
rosa create machinepool \  
  --cluster=my-rosa-cluster \  
  --name=gpu-pool \  
  --instance-type=g4dn.xlarge \  
  --replicas=1 \  
  --labels="node.kubernetes.io/workload-type=gpu" \  
  --taints="nvidia.com/gpu=present:NoSchedule"
```

Step 2: Deploy NVIDIA GPU Operator: Install the NVIDIA GPU Operator from OperatorHub to automatically manage drivers and CUDA runtimes.

Step 3: Schedule the GPU Workload:

None

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: gpu-inference-job  
  namespace: my-app  
spec:  
  nodeSelector:  
    node.kubernetes.io/instance-type: g4dn.xlarge  
  tolerations:  
  - key: "nvidia.com/gpu"  
    operator: "Exists"  
    effect: "NoSchedule"  
  containers:  
  - name: cuda-vector-add  
    image: "nvcr.io/nvidia/k8s/cuda-sample:vectoradd-cuda10.2"  
    resources:
```

```
limits:
  nvidia.com/gpu: 1
```

- Official Documentation & Guides:
 - [ROSA with NVIDIA GPU Workloads and OpenShift AI](#)

Scenario 6: Terraform Script for OpenShift Auth, Project Creation, and Helm Installation

1. Summary of Ask

Provide a Terraform (.tf) script that authenticates to the ROSA/OpenShift cluster, creates an OpenShift Project (namespace), and deploys a Helm chart.

2. Technical Solution (main.tf)

None

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.25.0"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 2.12.0"
    }
  }
}

# 1. Provider Setup using OpenShift API Server Token
provider "kubernetes" {
  host      =
  "https://api.my-rosa-cluster.openshiftapps.com:6443"
  token     = var.openshift_token
  insecure  = false
}
```

```
provider "helm" {
  kubernetes {
    host      =
"https://api.my-rosa-cluster.openshiftapps.com:6443"
    token     = var.openshift_token
    insecure  = false
  }
}

variable "openshift_token" {
  type        = string
  sensitive   = true
  description = "OpenShift OAuth ServiceAccount / Admin Token"
}

# 2. Create OpenShift Project (OpenShift CRD:
project.openshift.io/v1)
resource "kubernetes_manifest" "app_project" {
  manifest = {
    "apiVersion" = "project.openshift.io/v1"
    "kind"       = "Project"
    "metadata"   = {
      "name" = "marketing-apps"
      "labels" = {
        "environment" = "production"
      }
    }
  }
}

# 3. Deploy Helm Chart into the Created Project
resource "helm_release" "nginx_webserver" {
```

```
name      = "internal-webserver"
repository = "https://charts.bitnami.com/bitnami"
chart     = "nginx"
version   = "15.4.4"
namespace = "marketing-apps"

depends_on = [kubernetes_manifest.app_project]

set {
  name  = "replicaCount"
  value = "2"
}

set {
  name  = "service.type"
  value = "ClusterIP"
}
}
```

- Official Documentation & Guides:
 - [Deploying a ROSA HCP Cluster with Terraform](#)

Part 2: Non-Kubernetes Workloads

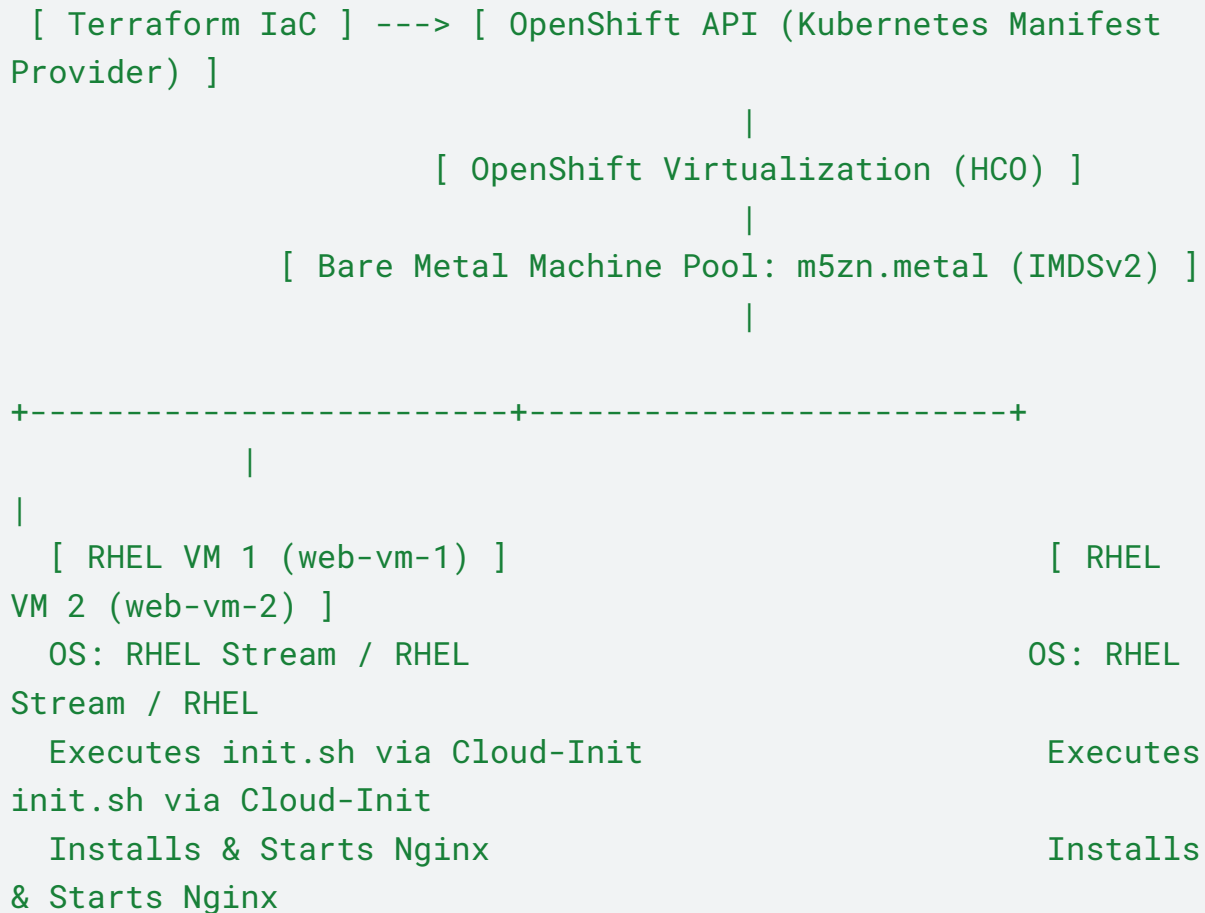
Scenario 7: Building RHEL VMs via OpenShift Virtualization, Terraform, and Cloud-Init (init.sh)

1. Summary of Ask

Demonstrate using ROSA with OpenShift Virtualization (KubeVirt) to run VM workloads alongside containers. Provision two Red Hat Enterprise Linux (RHEL) VMs via Terraform (kubernetes_manifest) without Ansible, utilizing cloud-init / userData (init.sh bash script) to install Nginx and write initial web content post-build.

2. Architecture Diagram

None



3. Prerequisites

OpenShift Virtualization requires worker nodes running on Bare Metal EC2 instances (e.g., `m5zn.metal` or `m5.metal`) with IMDSv2 metadata tokens required (`--ec2-metadata-http-tokens=required`).

4. Technical Solution (`vms.tf`)

None

```
# Terraform Configuration for 2 RHEL VMs on ROSA OpenShift
Virtualization

resource "kubernetes_manifest" "rhel_vm" {
  count = 2

  manifest = {
    "apiVersion" = "kubevirt.io/v1"
```

```
"kind"          = "VirtualMachine"
"metadata"      = {
  "name"         = "rhel-web-vm-${count.index + 1}"
  "namespace"    = "marketing-apps"
  "labels" = {
    "app" = "legacy-rhel-web"
  }
}
"spec" = {
  "running" = true
  "template" = {
    "metadata" = {
      "labels" = {
        "app" = "legacy-rhel-web"
      }
    }
  }
  "spec" = {
    "domain" = {
      "devices" = {
        "disks" = [
          {
            "name" = "bootdisk"
            "disk" = { "bus" = "virtio" }
          },
          {
            "name" = "cloudinitdisk"
            "disk" = { "bus" = "virtio" }
          }
        ]
      }
    }
    "resources" = {
      "requests" = {
        "memory" = "2Gi"
      }
    }
  }
}
```

```

        "cpu"      = "1"
    }
}
}
"volumes" = [
    # 1. Boot Volume from OpenShift Virtualization
    RHEL Golden Image
    {
        "name" = "bootdisk"
        "dataVolume" = {
            "dataVolumeTemplate" = {
                "metadata" = {
                    "name" = "rhel-disk-`${count.index + 1}`"
                }
            }
            "spec" = {
                "source" = {
                    "pvc" = {
                        "namespace" =
"openshift-virtualization-os-images"
                        "name"      = "rhel10" # Golden Image
DataSource
                    }
                }
            }
            "storage" = {
                "resources" = {
                    "requests" = {
                        "storage" = "20Gi"
                    }
                }
            }
            "storageClassName" = "gp3-csi"
        }
    }
}
}
}

```

```

    }
  },
  # 2. Cloud-Init Volume carrying the init.sh Bash
Script
  {
    "name" = "cloudinitdisk"
    "cloudInitNoCloud" = {
      "userData" = <<-EOF
        #!/bin/bash
        echo "Executing init.sh post-build
provisioning..."
        dnf install -y nginx
        systemctl enable --now nginx
        echo "<h1>Hello from RHEL VM ${count.index +
1} on ROSA Virt!</h1>" > /var/www/html/index.html
        systemctl restart nginx
      EOF
    }
  }
]
}
}
}
}
}
}
}
}
}
}

```

- Official Documentation & Tutorials:
 - [Deploying OpenShift Virtualization on ROSA](#)
 - [Running Virtual Machine on ROSA HCP Tutorial](#)
 - [Ingress to ROSA Virt VMs with Site-to-Site IPsec VPN](#)

💡 Next Steps: Would you like to explore configuring Certificate-Based Site-to-Site IPsec VPN with Libreswan for direct, non-NAT network routing into OpenShift Virtualization VM overlay networks, or examine GitOps (ArgoCD) patterns for managing these manifests across multi-region ROSA clusters?

Saved responses are view only

Gemini Notebook can be inaccurate; please double check its responses.